Software Project Management – IS212

Refactoring

Announcements

Announcements – Weeks 12-14

- No class in **Week 12** – use it to finish your final sprint & deliverables
- Deliverables must be submitted by Week 12 Friday @ 23:59
- Please take note of the slide template (released on eLearn) that you are to fill and include
- **Week 13 –** Closed-Door Q&A Session with Instructors
  - *Strict timing of 20 mins (schedule to be announced soon)*
  - *Sessions are private – only attend for your team's slot*
  - *Wait outside seminar room until called in (be there **5 mins before**)*
  - *Please be prepared to answer questions about your scrum process and software increment*
  - *Your instructors will ask you to demonstrate certain features of your software – **please be ready!***
  - *Expect us to pick a user story at random and ask you to trace it to its acceptance criteria, test cases, and code (which you must be accountable for)*
- **Week 14 –** mandatory intra-team peer evaluation due Week 14 Friday @ 23:59
  - *No late submissions possible. There is a penalty for non-submission*

Announcements – Final Exam

- **Thursday 26th November 2026, 13:00–15:00 (2 hrs)**
  - *Check OASIS for the room*
  - *Questions will be answered using your laptop with Respondus Lockdown Browser*
  - *Scrap paper provided; calculators allowed (but not needed)*

- Paper will consist of 80 marks
  - *Part A – quiz-style questions (40 marks)*
  - *Part B – written questions (40 marks)*

- We will share a sample exam paper by Week 14

- Clear and concise answers appreciated – don't 🧠-dump!

Announcements – Instructor Feedback

- Please can we have a moment of your time to complete the **UG Instructor Feedback** for IS212 😊

- (This is separate from the Faculty / Course Evaluation exercise.)









Recap

SMU Classification: Restricted

Week 10 Recap – Agentic AI

- Why has review become the delivery constraint for many teams?
- CI passed on PR #57. Why was that not enough evidence to approve it?
- What is the difference between guidance (e.g. AGENTS.md) and enforcement (e.g. a CI check)?
- Name and explain the four kinds of technical debt we discussed for the agentic era
- Which three factors influence the “delegation dial” for a task?

Today: Refactoring

Objectives

You will be able to:

- Explain why and when refactoring is worth the investment
- Recognise code smells, choose a suitable refactoring, and judge whether the result is better
- Explain how to gain confidence that a refactoring preserves behaviour – including with agent-made changes
- Decide, and justify, when to schedule refactoring work in a Scrum project



SMU Classification: Restricted

Refactoring

Why Refactor? Features vs. Futures

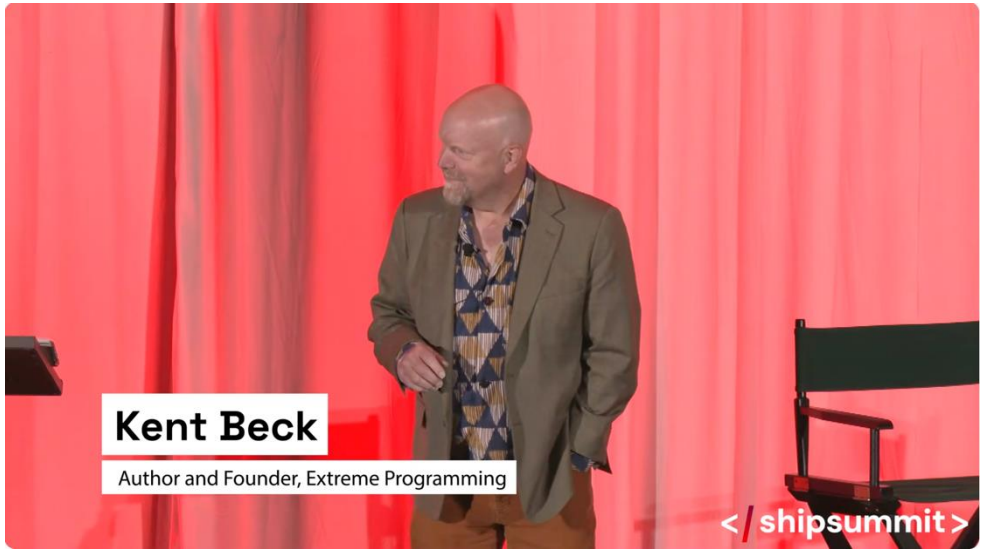
Kent Beck: software is valuable in **two** ways

Features – what the software does **today**

- Visible to users and stakeholders
- Easy to count, demo and plan for

Futures – what it could cheaply do **tomorrow**

- Options for changes you haven't needed yet
- Hard to see or measure, so easy to neglect



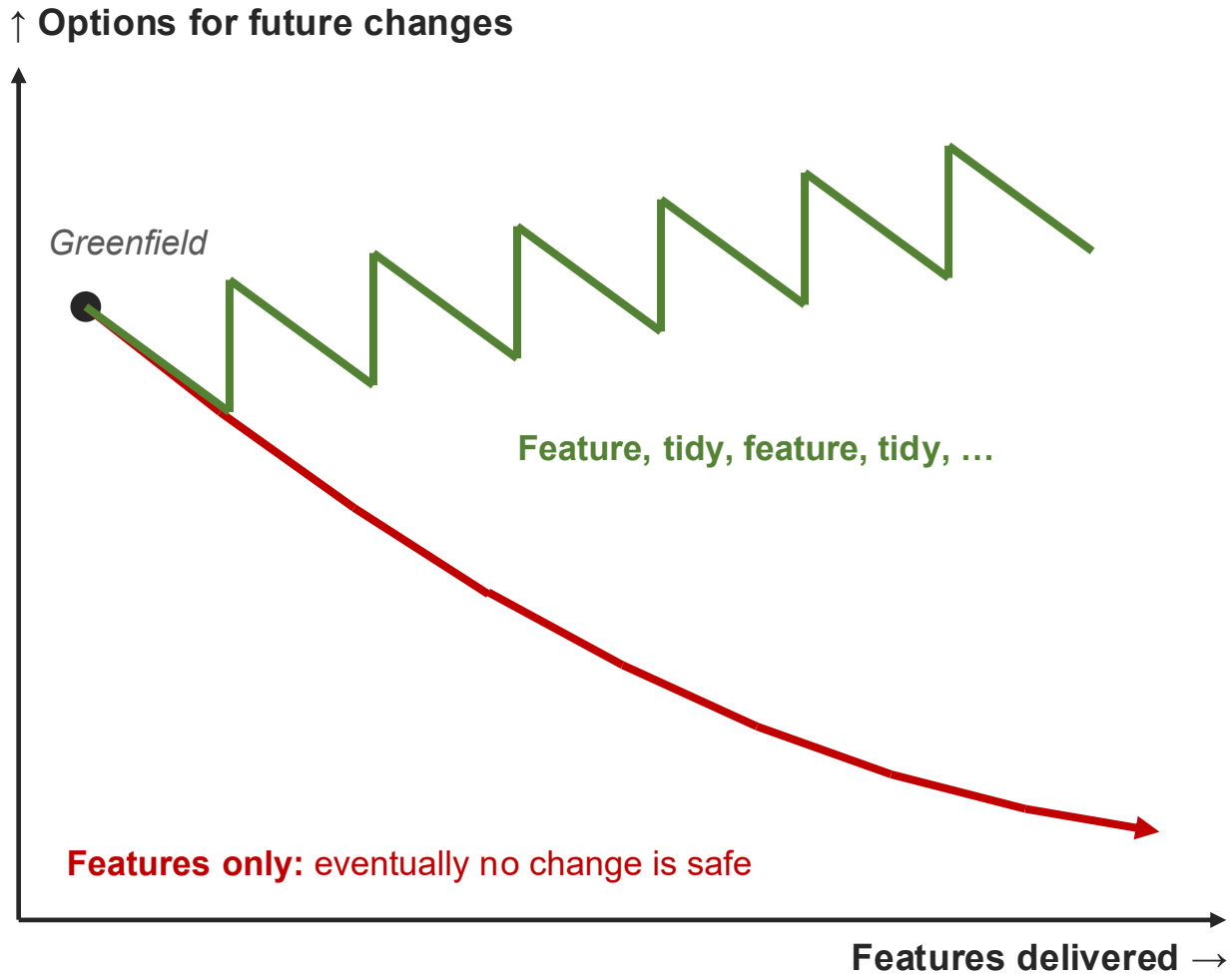
▶ Kent Beck – Features vs. Futures (YouTube)

Features tend to consume future options. **Tidying can restore or increase those options.**

Today: "refactoring" – improving structure without changing behaviour – is one way to make that investment.

Kent Beck, "Why Does Development Slow?" (Nov 2025); "Features Versus Futures", IT Revolution AI Summit (Spring 2026); Tidy First? (O'Reilly, 2023)

Why Does Development Slow Down?



Features often move right and down: they add value but tend to use up options

Tidying moves up: it can restore, or even increase, options

AI coding agents can speed up feature delivery, but without attention to options, development may hit its limits sooner

"Coding with a genie compresses time."

– Kent Beck

Adapted from Kent Beck, "Why Does Development Slow?", newsletter.kentbeck.com (2025). Illustrative geometry, not Beck's exact figure.

Make It Run, Then Make It Right



In the space between features, ask:

1. What is the next feature?
2. What in the current code **makes it harder**?
3. What could we tidy to **make it easier**?

"Make the change easy, then make the easy change." – Kent Beck

Leave time to <u>understand</u>

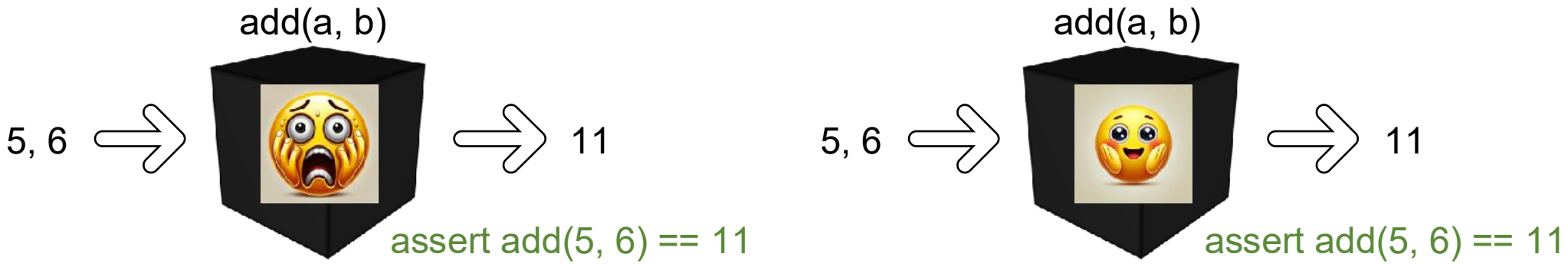
Agents can add features faster than a team can understand them.

Pacing the work leaves time to understand what has been built, otherwise it becomes **comprehension debt**.

As AI makes it easier to generate and modify code, **maintaining good structure** – and **verifying that changes preserve behaviour** – becomes increasingly important.

What is Refactoring?

- Refactoring is the process of making changes to a system such that:
  - *The **external behaviour** of the system does not change (what users and callers can observe)*
  - *The **internal structure** of the system is improved (e.g. easier to understand / modify)*



- Refactoring is not the same as debugging – the code already works, but the way it is structured can be improved

- Existing automated tests are thus a safety net as you go through the process of improving the code

Example

```
def apply_discount(price, discount_type):
    if discount_type == "student":
        return price - (price * 0.10)
    elif discount_type == "senior":
        return price - (price * 0.15)
    elif discount_type == "prof":
        return price - (price * 0.20)
    else:
        return price
```

```
assert apply_discount(50, "student") == 45
assert apply_discount(40, "prof") == 32
...
```



```
def apply_discount(price, discount_type):
    discounts = {
        "student": 0.10,
        "senior": 0.15,
        "prof": 0.20
    }
    discount = discounts.get(discount_type, 0)
    return price - (price * discount)
```

In what ways was the method **improved**?

Was This a Refactoring?

Recall PR #57 from Week 10 – the agent's **second** commit

```
772d7fd  Standardise error responses; update tests for new error format
app.py – handle_booking_error()
-    return jsonify({"error": str(err)}), err.status
+    return jsonify({"error": str(err), "code": err.code}), err.status
tests/test_booking_api.py – test_overlapping_booking_is_rejected
     assert resp.status_code == 409
-    assert resp.get_json() == {"error": "Room is not available"}
```



what about this?

What changed?

The agent added a `code` field to the JSON error response.

The tests passed **– but is this a refactoring?**

No – this changes externally observable behaviour.
The API response now has an additional field. This **may** be a useful improvement, but it could affect existing clients. Passing tests doesn't guarantee compatibility.

When to Refactor?



- ***Rule of Three –*** attributed to Don Roberts
  1. *Coding something for the first time?* ***Just get it done*** ✅
  2. *Something similar for the second time?* ***Cringe*** 😬***; do the same thing anyway***
  3. *Something similar for the third time?* ***Start refactoring*** 🧑‍🔧
- ***During a Code Review –*** last chance to 'tidy up' before merging
  - *With AI-generated code, review is an important opportunity to spot structural problems*
- ***When Fixing Bugs –*** why was the bug missed? Was the code hard to test? Inadequately tested? Hard to understand? Clean the code while fixing
- ***When Adding a Feature –*** adding new features typically involves extending an existing (often dirty!) code base

SMU Classification: Restricted

When Should We Refactor?

The decision: will the improvement justify the cost, risk, and review effort?

Code we're changing

Consider refactoring if the **existing structure** makes the **change harder**, riskier, or more expensive.

Refactor before or after the change, whichever makes more sense.

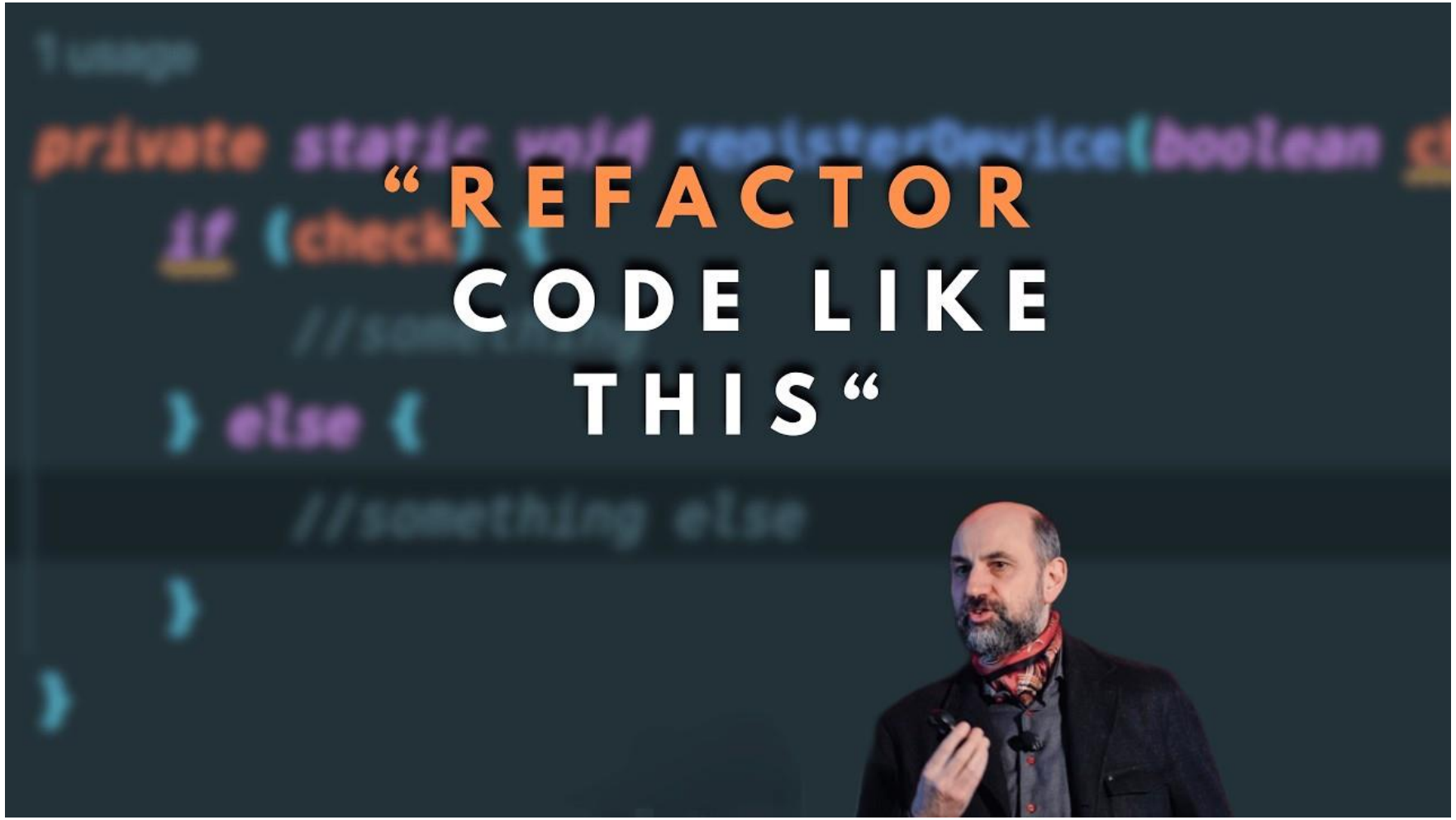
Code we're not changing

If it **works** and is **unlikely to change**, leave it alone.

Cleaner code alone may not justify the effort and risk.

With AI coding agents: Refactoring is cheap to propose, but **not free to verify**. Prefer focused improvements that make an anticipated change easier.

Refactoring in Small Steps – *Even with AI*



https://www.youtube.com/watch?v=naYIFpHDp_s

How to Refactor?

- Refactoring should be done as a series of small changes
  - *Commit frequently so that you can easily "roll back"*
- The code should become "cleaner"
  - *i.e. easier to understand, test, or (safely) modify*

What is **"clean code"**?

- Keep structural and behavioural changes in separate commits – no new functionality while refactoring
- Run automated tests after each refactoring step
  - *Tests of externally observable behaviours should continue to pass*
  - *Tests coupled to implementation details may need updating, but the expected behaviour must remain unchanged*



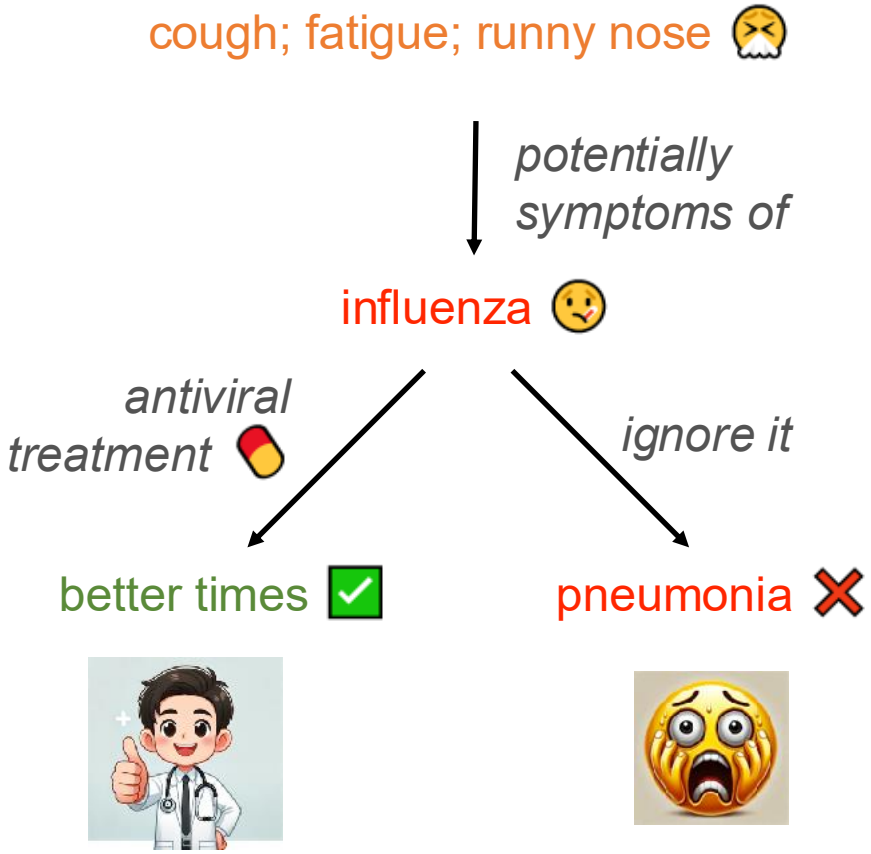
GitHub Actions

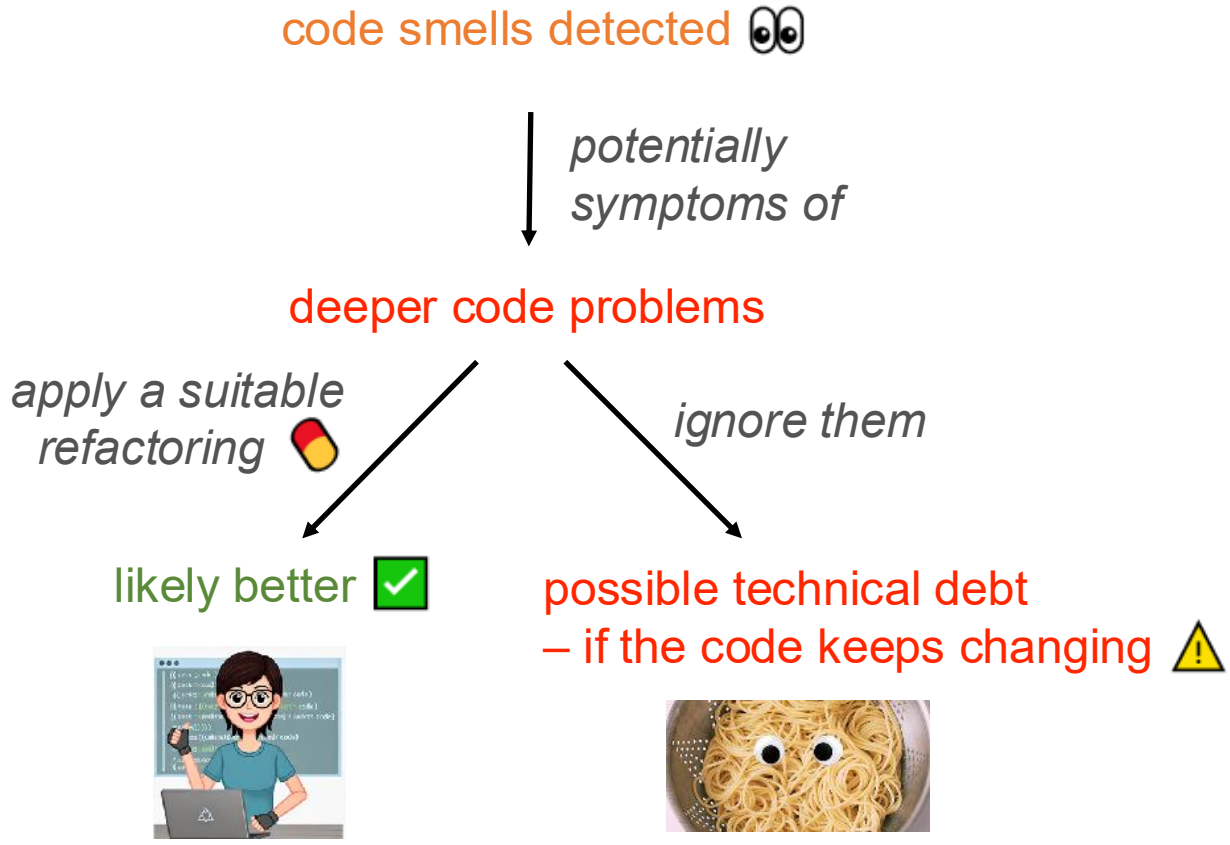
- **With AI coding agents:** prefer small, focused, reviewable changes over large refactoring PRs. Verify each step rather than relying solely on the agent's claim that behaviour was preserved.

Code Smells 

Code Smells are “Symptoms”

- Code smells are symptoms of potentially deeper code problems – clues to investigate, not proof of bad design





SMU Classification: Restricted

Code Smells – Five Families

Fowler's Refactoring catalogues around two dozen smells. A common grouping:

Bloaters

Code that has grown too large to work with

e.g.

- **Long Method**
- **Data Clumps**
- Long Parameter List
- Primitive Obsession

OO Abusers

OO features misused – or not used where they would help

e.g.

- **Switch Statements**
- Refused Bequest
- Temporary Field

Change Preventers

One change forces edits in many places

e.g.

- **Shotgun Surgery**
- Divergent Change

Dispensables

Code you could remove without losing anything

e.g.

- **Duplicate Code**
- Dead Code
- Speculative Generality

Couplers

Too much interdependence between parts

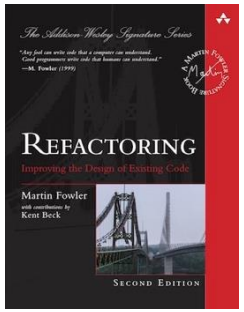
e.g.

- **Feature Envy**
- Inappropriate Intimacy
- Message Chains

Bold = the six we'll find in the booking app next

Focus on **recognising code smells**, understanding why they can be problematic, and identifying **suitable refactorings**.



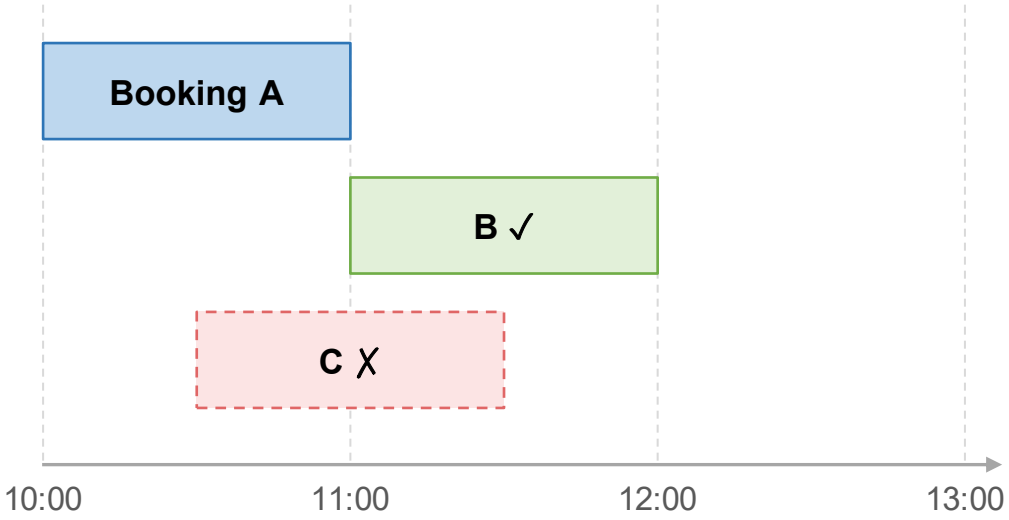


Martin Fowler, Refactoring (2nd ed., Addison-Wesley, 2018); grouping from refactoring.guru/refactoring/smells

Running Example – Study-Room Booking

1. A booking lasts **at most 2 hours**
2. Rooms can be booked **up to 7 days ahead**
3. **Back-to-back** bookings are fine; **overlapping** ones are not
4. Cancelling **less than 2 h before the start** records a late-cancellation strike

Rule #3 on one room – GSR 2-01, 20 Oct



B: back-to-back with A – allowed. C: overlaps A – rejected.

Smells – Duplication & Complexity

Long Method
Bloater

One function handles parsing, validation, and saving

```
def create_booking(room_id, student_id, start, end):
    start, end = parse_slot(start), parse_slot(end)
    if end - start > MAX_DURATION: ...           # rule 1
    clash = Booking.query.filter(...).first()    # rule 3
    ...   # then build, add and commit: 16 lines in all
```

→ *Remedy* **Extract Method**

Duplicate Code
Dispensable

Validation rules repeated across functions; two date parsers

```
# create_booking()
    if end - start > MAX_DURATION: ...
# extend_booking()
    if end - booking.start > MAX_DURATION: ...
# ...and the clash query; parse_slot() vs _to_datetime()
```

→ *Remedy* **Extract Method**

Data Clumps
Bloater

start and end repeatedly passed together

```
def create_booking(room_id, student_id, start, end): ...
def _check_duration(start, end): ...
def _check_room_free(room_id, start, end, ignore_id=None): ...
```

→ *Remedy* **Replace Data Value with Object**

*Identifying a code smell does **not** automatically mean refactoring is necessary. Consider whether the improvement justifies the effort and risk.*

Smells – Coupling & Change

Shotgun Surgery
Change Preventer

Changing the 2-hour limit means edits in several files

```
# booking_service.py – create_booking()
    if end - start > MAX_DURATION: ...
# booking_service.py – extend_booking()
    if end - booking.start > MAX_DURATION: ...
# index.html: <li>A booking lasts at most 2 hours</li>
```

→ *Remedy*
Extract / Move Method
one home for each rule

Feature Envy
Coupler

A function that only uses another object's data

```
# booking_service.py
def is_late_cancellation(booking, now):
    return booking.start - now < LATE_CANCEL_WINDOW
```

→ *Remedy*
Move Method

Switch Statements
OO Abuser

Behaviour chosen by checking a type code

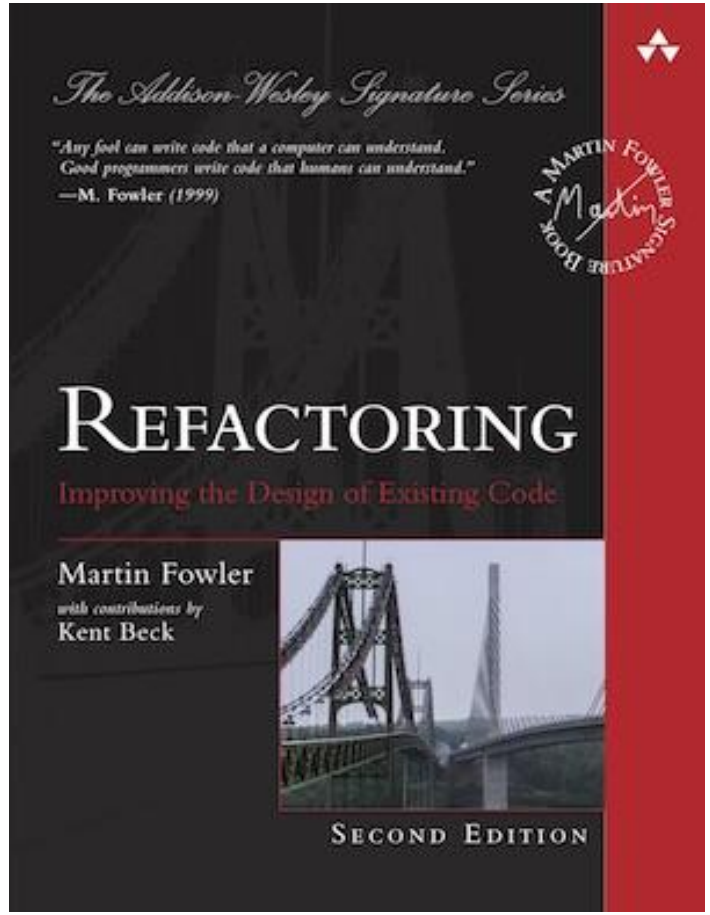
```
if room_type == "gsr": ...
elif room_type == "seminar": ...
elif room_type == "pod": ...
```

→ *Remedy*
Replace Conditional with Polymorphism
…if the design needs it

Refactoring Patterns 💊

If Code Smells are the Symptoms…

- …then refactoring patterns are the medicine



- Martin Fowler identified around seventy refactorings in his book

- Five examples (booking app):
  1. *Extract Method*
  2. *Replace Temp with Query*
  3. *Replace Data Value with Object*
  4. *Replace Conditional with Polymorphism*
  5. *Move Method*



Refactoring Pattern – Extract Method

Extract Method – move a block of code into a well-named **function of its own**

Can deodorise: Long Method, Duplicate Code

Before – booking_service.py

```
def create_booking(room_id, student_id, start, end):
    start, end = parse_slot(start), parse_slot(end)
    if end - start > MAX_DURATION:
        raise BookingError("Bookings are limited to 2 hours")
    clash = Booking.query.filter(
        Booking.room_id == room_id,
        Booking.start < end,
        Booking.end > start,
    ).first()
    if clash:
        raise BookingError("Room is not available", status=409)
    booking = Booking(room_id=room_id, student_id=student_id,
                      start=start, end=end)
    db.session.add(booking)
    db.session.commit()
    return booking
```

After

```
def create_booking(room_id, student_id, start, end):
    start, end = parse_slot(start), parse_slot(end)
    _check_duration(start, end)
    _check_room_free(room_id, start, end)
    booking = Booking(room_id=room_id, student_id=student_id,
                      start=start, end=end)
    db.session.add(booking)
    db.session.commit()
    return booking

def _check_duration(start, end):
    if end - start > MAX_DURATION:
        raise BookingError("Bookings are limited to 2 hours")

def _check_room_free(room_id, start, end, ignore_id=None):
    ...  # the clash query, now shared with extend_booking()
```

`extend_booking()` contained the same two blocks – now both functions call the helpers, and the names say what the rules are

Refactoring Pattern – Replace Temp with Query

Replace Temp with Query – replace a local variable that holds an expression with a **call to a query function**

Can deodorise: Long Method, Duplicate Code (the query can be reused)

Before – the "cancel a booking" story (rule 4)

```
def cancel_booking(booking, now):
    late = booking.start - now < timedelta(hours=2)
    if late:
        record_strike(booking.student_id)
    booking.status = "cancelled"
    db.session.commit()

def cancellation_warning(booking, now):
    late = booking.start - now < timedelta(hours=2)
    if late:
        return "Cancelling now will record a strike"
    return None
```

After

```
def cancel_booking(booking, now):
    if is_late_cancellation(booking, now):
        record_strike(booking.student_id)
    booking.status = "cancelled"
    db.session.commit()

def cancellation_warning(booking, now):
    if is_late_cancellation(booking, now):
        return "Cancelling now will record a strike"
    return None

def is_late_cancellation(booking, now):
    return booking.start - now < LATE_CANCEL_WINDOW
```

Not always an improvement

If the temp holds something expensive or changeable – e.g. `Booking.query…all()` – a query function runs it again on every call, and may not return the same answer twice.

Refactoring Pattern – Replace Data Value with Object

Replace Data Value with Object – turn primitives that travel together into **an object with behaviour**

Can deodorise: Primitive Obsession, Data Clumps

Before

```
# start and end travel together everywhere…
def create_booking(room_id, student_id, start, end): ...
def extend_booking(booking_id, new_end): ...
def _check_duration(start, end): ...
def _check_room_free(room_id, start, end, ignore_id=None): ...

start, end = parse_slot(start), parse_slot(end)
if end - start > MAX_DURATION: ...
# and rule 3 is buried in a database query:
Booking.start < end, Booking.end > start
```

Discuss

- Time-related concepts now have a clear home, instead of separate primitives throughout the code
- But the database still stores two columns – where does the object stop being useful?

After

```
@dataclass(frozen=True)
class TimeSlot:
    start: datetime
    end: datetime

    @classmethod
    def parse(cls, start, end):
        return cls(parse_slot(start), parse_slot(end))

    @property
    def duration(self):
        return self.end - self.start

    def overlaps(self, other):    # rule 3, named
        return (self.start < other.end
                and other.start < self.end)

slot = TimeSlot.parse(body["start"], body["end"])
if slot.duration > MAX_DURATION: ...
```

Refactoring Pattern – Replace Conditional with Polymorphism

Replace Conditional with Polymorphism – when behaviour varies by type, give **each type its own class**

Can deodorise: Switch Statements

```python
def check_policy(room_type, slot, now):
    if room_type == "gsr":
        if slot.duration > timedelta(hours=2):
            raise BookingError("Max 2 hours")
    elif room_type == "seminar":
        if slot.duration > timedelta(hours=3):
            raise BookingError("Max 3 hours")
        if slot.start - now < timedelta(days=1):
            raise BookingError("Book a day ahead")
    elif room_type == "pod":
        if slot.start.date() != now.date():
            raise BookingError("Same day only")
```



```python
class GsrPolicy:
    def check(self, slot, now):
        if slot.duration > timedelta(hours=2):
            raise BookingError("Max 2 hours")

class SeminarPolicy:
    def check(self, slot, now):
        if slot.duration > timedelta(hours=3):
            raise BookingError("Max 3 hours")
        if slot.start - now < timedelta(days=1):
            raise BookingError("Book a day ahead")

class PodPolicy:
    def check(self, slot, now):
        if slot.start.date() != now.date():
            raise BookingError("Same day only")

POLICIES = {"gsr": GsrPolicy(), "seminar": SeminarPolicy(),
            "pod": PodPolicy()}
POLICIES[room_type].check(slot, now)
```

Worth it only if the design needs it

If rooms differed only in maximum hours, a dictionary (like the discount example) would be simpler.

Spot the difference?

Behaviour-preserving? Try `"lab"` as the room type in both versions.

Refactoring Pattern – Move Method

Move Method – move a function to the class whose data it uses most

Can deodorise: Feature Envy

Before – from the previous refactoring

```
# booking_service.py
def is_late_cancellation(booking, now):
    return booking.start - now < LATE_CANCEL_WINDOW

if is_late_cancellation(booking, now):
    record_strike(booking.student_id)
```

Uses only `booking`*'s data – it "envies" the Booking class*

After

```
# models.py
class Booking(db.Model):
    ...
    def is_late_cancellation(self, now):
        return self.start - now < LATE_CANCEL_WINDOW

# booking_service.py
if booking.is_late_cancellation(now):
    record_strike(booking.student_id)
```

A Code Smell Does Not Always Require Refactoring

`booking_to_json(b)` in app.py also uses only the booking's fields. Should it move to `Booking`? Is turning a booking into JSON the model's job – or the API layer's?

Does this Refactoring Improve the Code?

An agent was asked to *"make extend_booking() more readable"*:

```
def extend_booking(booking_id, new_end):
    booking = _load(booking_id)
    end = _parse(new_end)
    _validate(booking, end)
    _apply(booking, end)
    _save()
    return booking

def _load(booking_id):
    return db.get_or_404(Booking, booking_id)

def _parse(value):
    return _to_datetime(value)

def _save():
    db.session.commit()
# ... _validate() and _apply() follow
```

What are the limitations of this refactoring?

- One-line wrappers (`_load, _parse, _save`) introduce extra indirection with little benefit.
- Vague names (`_validate, _apply`) make it difficult to identify the booking rules.
- Suppose staff can extend bookings to 3 hours. Which function would need modifying?

A useful refactoring should make the code easier to understand, maintain, or modify **– not merely divide it into more functions**

When Should We Refactor?

Scenario

Next sprint brings a **new rule (staff may book up to 3 hours)** and finally **enforces an existing one** (rule 2: book at most 7 days ahead). The duration and availability checks are duplicated in `create_booking()` and `extend_booking()`, and a cancel endpoint is planned.

Refactor now?
before the new rules

Defer?
a backlog item, with a reason

Leave it alone?
accept the duplication

How would you address the duplication? Consider:

Anticipated changes
What is coming next, and how sure are we?

Expected future cost
What will the duplication cost each time?

Current risk
What could break, and how would we know?

Available capacity
Including time for testing and review



https://refactoring.guru/refactoring/techniques

SMU Classification: Restricted

Refactoring Safely with Tools & Agents

Tools: Degrees of Automation

One principle – **establish that behaviour is preserved** – with different degrees of automation and assurance

IDE refactorings

e.g. VS Code: Rename Symbol, Extract Method / Variable

Automated structural transformations, generally designed to preserve behaviour

Limits: Can miss dynamic references – in Flask: `url_for("…")` endpoint names, Jinja templates, `getattr()`

Preview the change before applying

Codemods

e.g. LibCST, ast-grep

Repeatable, rule-based transformations that can be inspected and tested

Limits: A rule can be right for most inputs and wrong for some – test the rule, then review a sample of its changes

Useful for the same change across many files

AI agents

e.g. Copilot (agent mode), Claude Code

Flexible transformations that can span many files and explain themselves

Limits: May also change behaviour, tests or scope (while claiming not to)

Need the most careful verification



More flexible transformations generally need more independent verification

Reviewing an Agent's Refactoring

Red flags in a "refactoring" PR

- Behavioural tests **weakened**, **deleted**, or **rewritten** to accommodate the change.
- **New behaviour:** extra validation, new messages, different accepted inputs
- Structural and behavioural changes **mixed** in one commit
- Two helpers that were only *nearly* the same, merged into one
- Claiming "no behaviour changes" without sufficient evidence beyond the existing tests
- A diff **too large** to review properly

Ask the agent to…

- Treat it as a **pure refactoring** – say so in the prompt
- Work in small steps and run the tests after each one
- Leave tests unchanged; stop and explain if one looks wrong
- Stop and ask when similar code behaves differently
- Put **any behaviour change** in a separate, labelled commit

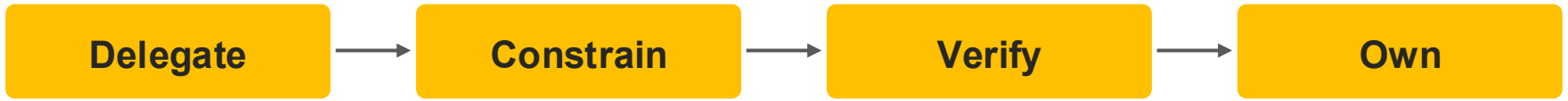
Week 11 – Three Questions

1 Why and when is refactoring worth doing?
Features vs. futures; tidy first, after, later – or never

2 What structural change would improve this code – and why?
Smells as clues; refactorings as moves; judge by the next change

3 How do we know the change hasn't altered behaviour?
Tests, small steps, careful review

And if you delegate refactoring to an agent:



SMU Classification: Restricted

Refactoring in the IS212 Project

- ***"Do I need to include evidence of refactoring in the IS212 project?"***

- **No.** Refactoring is not a required deliverable

- If your code already works and is reasonably well-structured, there's no need to refactor it

- If refactoring would make an upcoming change easier or safer, by all means do it!

- Don't refactor just to tick a box. You already have enough boxes to tick. 😊

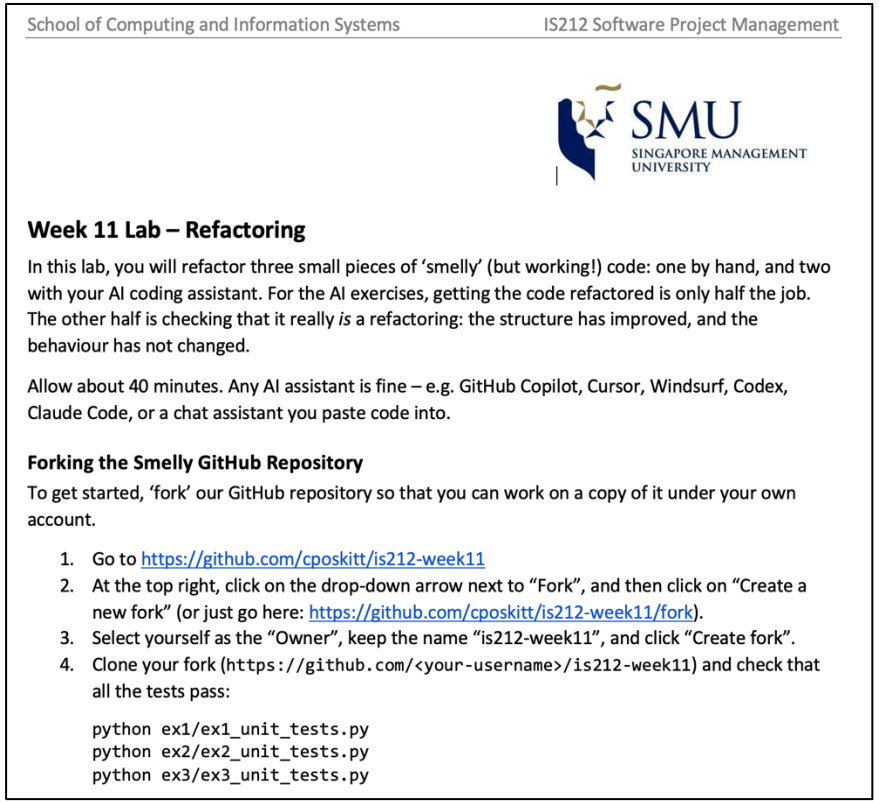


SMU Classification: Restricted

Lab & Reflections

Lab – Refactoring

- **Three exercises** to help you identify smells and apply refactorings
- Try with **hand** as well as with AI
- Formative (no submission) – for learning and to help prepare for the exam



School of Computing and Information Systems — IS212 Software Project Management

SMU SINGAPORE MANAGEMENT UNIVERSITY

Week 11 Lab – Refactoring

In this lab, you will refactor three small pieces of 'smelly' (but working!) code: one by hand, and two with your AI coding assistant. For the AI exercises, getting the code refactored is only half the job. The other half is checking that it really *is* a refactoring: the structure has improved, and the behaviour has not changed.

Allow about 40 minutes. Any AI assistant is fine – e.g. GitHub Copilot, Cursor, Windsurf, Codex, Claude Code, or a chat assistant you paste code into.

Forking the Smelly GitHub Repository

To get started, 'fork' our GitHub repository so that you can work on a copy of it under your own account.

1. Go to https://github.com/cposkitt/is212-week11
2. At the top right, click on the drop-down arrow next to "Fork", and then click on "Create a new fork" (or just go here: https://github.com/cposkitt/is212-week11/fork).
3. Select yourself as the "Owner", keep the name "is212-week11", and click "Create fork".
4. Clone your fork (`https://github.com/<your-username>/is212-week11`) and check that all the tests pass:

```
python ex1/ex1_unit_tests.py
python ex2/ex2_unit_tests.py
python ex3/ex3_unit_tests.py
```

Reminder – *Reflections*

- Please start filling in your weekly reflections now
- Complete them by two days after this class so that we have time to read them before we next meet 😊
- **http://smu.sg/is212-reflections-2026**



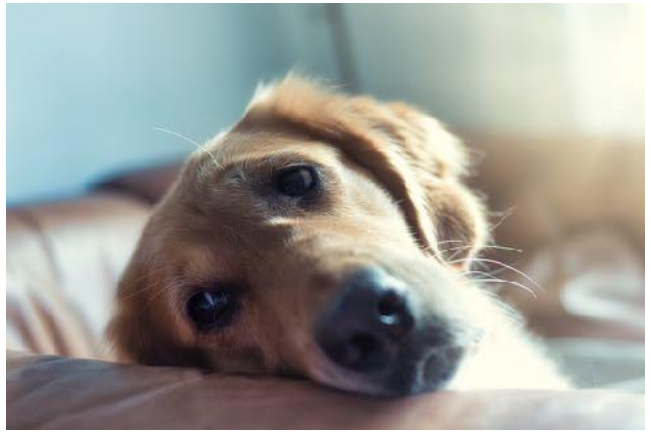




IS212 Wrap Up

software development processes, software quality, collaboration tools, project design & delivery

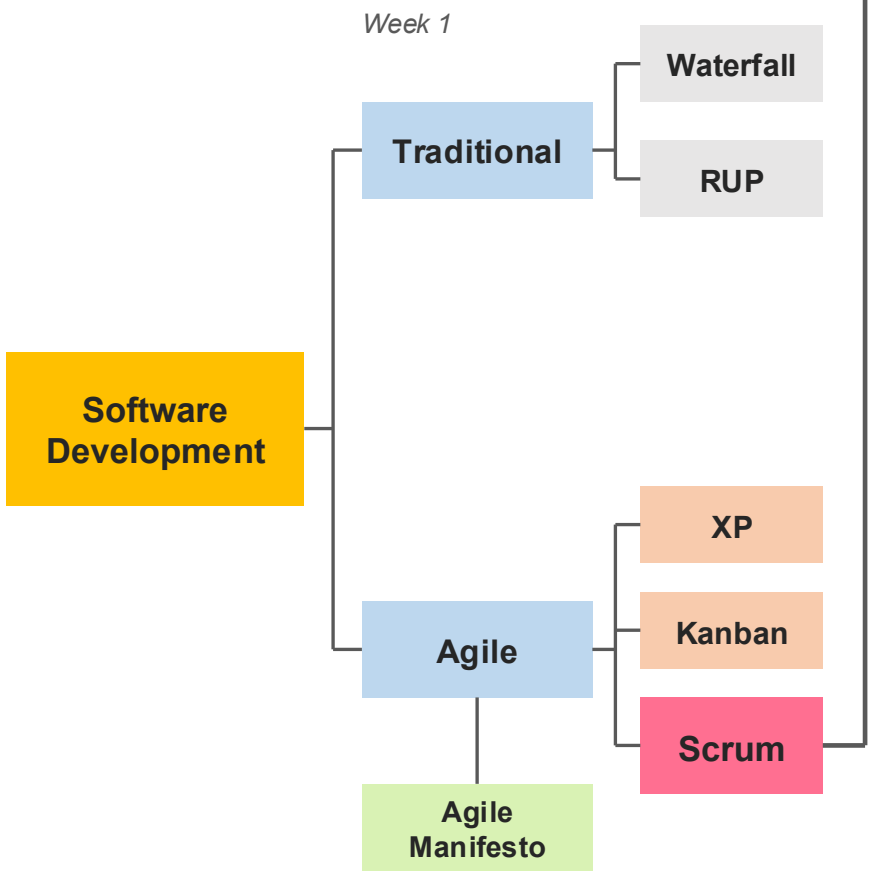




SMU Classification: Restricted

IS212 at a glance

AY2026-27 · Weeks 1–11



Plan & Specify

Weeks 1–4

Scrum roles & events W1, W3
- Sprint planning · stand-up
- Review · retrospective

User stories W2
- Epics · INVEST
- Story maps · splitting

Acceptance criteria W2, W4
- → test cases: +ve, −ve, boundary

Estimation W3
- Story points · planning poker
- Velocity & flow metrics · burndown

Design & Collaborate

Weeks 4, 5, 11

Architecture W4
- C4 model · "just enough" docs

Code-level design W5
- UML class & sequence diagrams

Collaborating with Git W5
- Branches · pull requests
- Feature-based vs trunk-based

Refactoring W11
- Smells
- Refactorings

Verify & Deliver

Weeks 6, 9, 10

Unit tests & TDD W6
- FIRST · coverage
- Red-green-refactor

Integration & E2E W9
- Test doubles · test data
- Flaky tests

Continuous integration W6
- GitHub Actions · status checks

Continuous delivery W9
- Release gates
- Delivery vs deployment

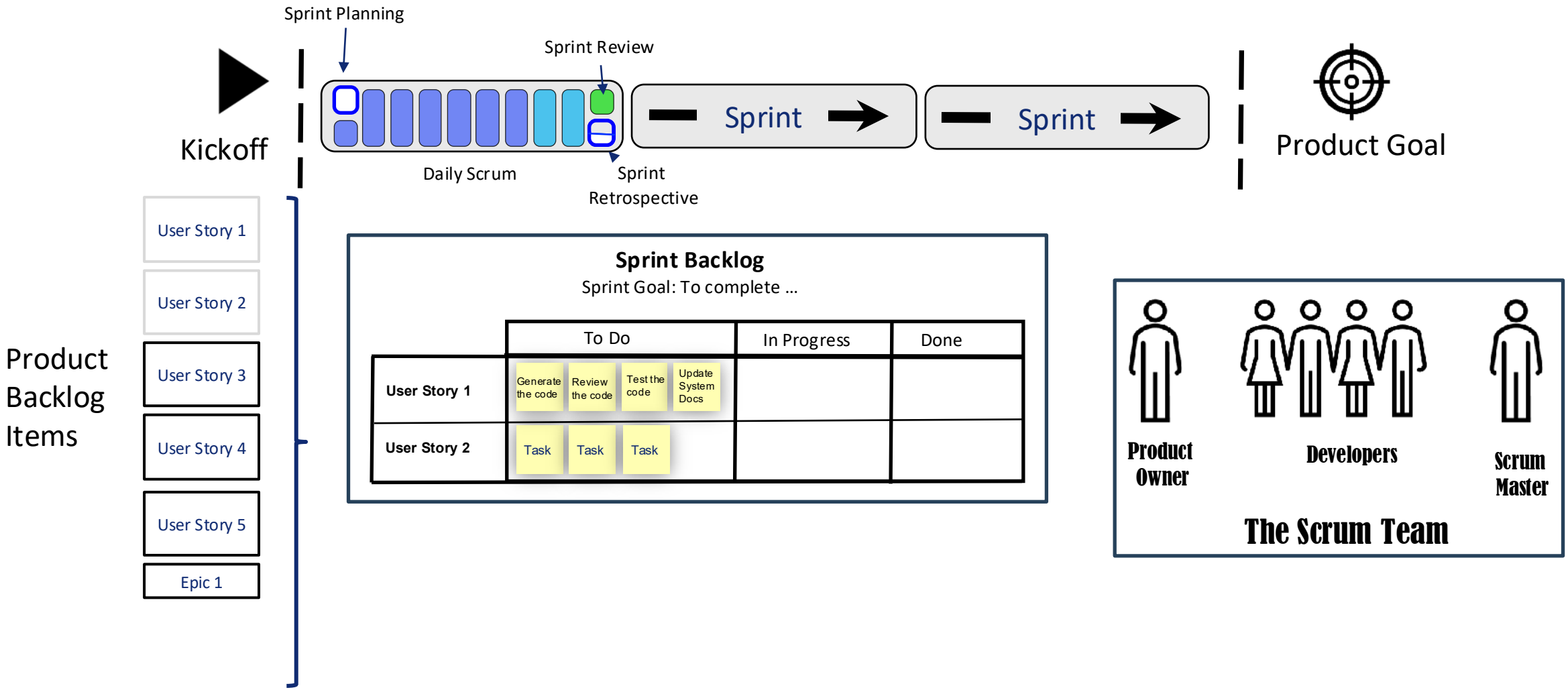
Review & tech debt W10
- Review as the bottleneck
- Agentic technical debt

AI & agents – in every week

Delegate → Constrain → Verify → Own

AI-drafted stories (W2) · AI in test design & C4 (W4) · Agents in Git workflows (W5)
Judging generated tests (W6, W9) · Reviewing agent PRs (W10) · Agent refactoring (W11)

*When code is cheap to generate, **trustworthy verification and accountability** are the scarce skills.*


Sprint Planning
Sprint Review
Kickoff
Daily Scrum
Sprint Retrospective
Sprint
Sprint
Product Goal
Product Backlog Items
User Story 1
User Story 2
User Story 3
User Story 4
User Story 5
Epic 1
Sprint Backlog
Sprint Goal: To complete ...
To Do
In Progress
Done
User Story 1
Generate the code
Review the code
Test the code
Update System Docs
User Story 2
Task
Task
Task
Product Owner
Developers
Scrum Master
The Scrum Team

SMU Classification: Restricted

Thank you! 